

Tarea 2: Iteraciones con purrr

Indicaciones generales

En los siguientes ejercicios se integran las herramientas vistas en la Unidad III:

- bucles `for`;
- funciones de orden superior;
- `map()`, `map_dbl()`, `map_chr()`, `map_lgl()`;
- `safely()`, `possibly()` y `quietly()`;
- `map2()` y `pmap()`;
- `keep()`, `discard()`, `some()`, `every()`, `detect()`, `reduce()` y `accumulate()`.

Para resolverlos debes cargar los paquetes:

```
library(purrr)
library(dplyr)
```

```
##
## Adjuntando el paquete: 'dplyr'

## The following objects are masked from 'package:stats':
##
##   filter, lag

## The following objects are masked from 'package:base':
##
##   intersect, setdiff, setequal, union
```

```
library(tibble)
```

Ejercicio 1

Simula datos normales usando distintas combinaciones de media, desviación estándar y tamaño de muestra.

Crea el siguiente data frame de parámetros: Luego:

1. Usa `pmap()` para generar las simulaciones.
2. Calcula la media observada de cada simulación.
3. Calcula la desviación estándar observada de cada simulación.
4. Guarda todo en un nuevo data frame.

```
parametros <- tibble(
  mean = c(0, 5, 10, 100),
  sd = c(1, 2, 5, 10),
  n = c(10, 20, 30, 40)
)
```

```
simulaciones <- pmap(parametros,rnorm)

medias <- map_dbl(simulaciones,mean)
desviaciones <- map_dbl(simulaciones,sd)

parametro2 <- mutate(parametros,medias,desviaciones)
parametro2
```

```
## # A tibble: 4 x 5
##   mean    sd    n medias desviaciones
##   <dbl> <dbl> <dbl>   <dbl>         <dbl>
## 1     0     1   10  0.0463         0.949
## 2     5     2   20  4.84          1.45
## 3    10     5   30  8.81          5.18
## 4   100    10   40 99.0           8.28
```

Ejercicio 2

Usa la base de datos `mtcars`.

Haz lo siguiente:

1. Selecciona únicamente las columnas numéricas de `mtcars`.
2. Crea una lista de funciones resumen con:
 - media;
 - mediana;
 - desviación estándar;
 - mínimo;
 - máximo;
 - rango, definido como máximo menos mínimo.
3. Aplica cada función resumen a cada columna numérica.
4. Construye un data frame donde:
 - cada fila sea una variable;
 - cada columna sea una medida resumen.
5. Además, identifica:
 - si todas las columnas seleccionadas son numéricas;
 - si alguna columna tiene desviación estándar mayor que 100;
 - cuál es la primera variable cuyo rango es mayor que 300.

```
mtcars_numeric <- keep(mtcars,is.numeric)
rango <- (function(x){
  max(x) - min(x)
})
funciones_resumen <- list(media = mean, mediana = median,desviacion_estandar = sd,minimo = min,maximo =
multi_funcion <- function(datos, funcion){
```

```

salida <- data.frame(matrix(
  nrow = length(datos),
  ncol = length(funcion)
))

for(j in seq_along(funcion)){
  salida[[j]] <- map_dbl(datos, funcion[[j]])
}
salida
}

resultado <- multi_funcion(mtcars_numeric,funciones_resumen)
names(resultado) <- c(names(funciones_resumen))
rownames(resultado) <- c(names(mtcars_numeric))
resultado

```

```

##          media mediana desviacion_estandar minimo maximo  rango
## mpg    20.090625  19.200          6.0269481 10.400  33.900  23.500
## cyl      6.187500   6.000          1.7859216  4.000   8.000   4.000
## disp  230.721875 196.300        123.9386938 71.100 472.000 400.900
## hp    146.687500 123.000        68.5628685 52.000 335.000 283.000
## drat    3.596563   3.695          0.5346787  2.760   4.930   2.170
## wt      3.217250   3.325          0.9784574  1.513   5.424   3.911
## qsec   17.848750  17.710          1.7869432 14.500  22.900   8.400
## vs      0.437500   0.000          0.5040161  0.000   1.000   1.000
## am      0.406250   0.000          0.4989909  0.000   1.000   1.000
## gear    3.687500   4.000          0.7378041  3.000   5.000   2.000
## carb    2.812500   2.000          1.6152000  1.000   8.000   7.000

```

```

print(paste("todas las columnas son numericas",every(resultado,is.numeric)))

```

```

## [1] "todas las columnas son numericas TRUE"

```

```

print(paste("alguna columna tiene desviación estándar mayor que 100",some(resultado$desviacion_estandar

```

```

## [1] "alguna columna tiene desviación estándar mayor que 100 TRUE"

```

```

print(paste("primera variable cuyo rango es mayor que 300 es ",rownames(resultado)[detect_index(resulta

```

```

## [1] "primera variable cuyo rango es mayor que 300 es  disp  con rango 400.9"

```